HPC FINAL LAB PROJECT · P08

High-Performance Conjugate-Gradient Benchmark

Project explanation & viva handbook — what the project does, how it works, what we found and why, plus 54 viva questions with model answers.

INSIDE

1 · The project in 60 seconds
2 · HPL vs HPCG — why this benchmark exists
3 · The maths: Poisson matrix, CSR, Conjugate Gradient
4 · Sequential, OpenMP, MPI & hybrid versions
5 · Results, graphs and the memory wall
6 · Glossary, numbers to memorise, 54 viva Q&As



2.1 M
equations solved (128^3 grid)

1.40×
best speedup (8 threads) = memory limit

1.69×
halo exchange vs Allgatherv (8 procs)

10 / 10
automated tests passing · 50 configs measured

HPCG-inspired educational implementation — not the official HPCG benchmark

github.com/Anuttaman216/HPC_FInal_Lab_Project

Contents

1 The project in 60 seconds
2 Background: HPL vs HPCG
3 The problem we solve
4 The Conjugate Gradient method
5 Four implementations
6 How MPI processes communicate
7 How we measured
8 Results
9 Why the speedup is limited
10 Tools, tests & files
11 Limitations & future work
12 Glossary
13 Numbers to memorise
14 Viva questions & answers

1 The project in 60 seconds

The whole story

Supercomputers are ranked using **HPL**, a test that mostly measures how fast a processor does arithmetic. Real scientific programs (weather, fluid flow, engineering) are slowed down by **moving data from memory** instead. **HPCG** was created in 2013 to measure that realistic behaviour. We built a smaller, educational version of HPCG: a **Conjugate Gradient** solver for over 2 million sparse equations, written four ways — **sequential, OpenMP, MPI and hybrid** — and measured how each one scales.

Finding 1 — the memory wall

8 cores were only **1.40×** faster than 1 core. Memory bandwidth grows only **1.32×** from 1 to 8 cores (21.9 → 28.9 GB/s). CG needs about 7.5 bytes from memory per arithmetic operation, so the cores mostly wait for data.

Finding 2 — communication design matters

With 8 MPI processes, sending the whole vector (**Allgatherv**) took 51% of the time and was slower than 1 core. Sending only the needed boundary data (**halo**) took 11.5% and was **1.69×** faster.

One-sentence conclusion: *for sparse scientific codes like CG, bytes per second matter more than flops per second — which is why HPCG complements HPL for ranking supercomputers.*

Everyday analogies

Memory-bound

A kitchen with 8 chefs but one narrow door for ingredients. Adding chefs does not help once the door is the bottleneck.

OpenMP vs MPI

OpenMP: several people working at the **same desk** (shared memory). MPI: people at **separate desks** passing notes (messages).

Conjugate Gradient

Improving a guess round after round, each round moving in a smarter direction, until the error is below 10^{-8}.

Halo exchange

Each worker asks only the neighbours for the edge of their part, instead of photocopying the whole book for everyone.

2 Background: HPL vs HPCG

HPC (High-Performance Computing) means many processors working together to solve problems faster (strong scaling) or to solve bigger problems (weak scaling). The **TOP500** list ranks supercomputers twice a year using **HPL**.

Aspect	HPL (LINPACK)	HPCG	This project
Problem	Dense random matrix	Sparse 27-point 3D stencil	Sparse 7-point 3D stencil
Method	LU factorisation	CG + multigrid preconditioner	Plain CG
Main kernel	Matrix × matrix (DGEMM)	Sparse matrix × vector, Gauss-Seidel	Sparse matrix × vector, dot products
Limited by	Arithmetic units (compute-bound)	Memory bandwidth, network	Memory bandwidth, communication
Typical % of peak	High	A few percent	Not measured (peak unknown)
Ranking	TOP500	HPCG list	**None — educational**

Why HPCG exists

Dongarra and Heroux (2013) argued that HPL rewards machines for arithmetic speed, while real applications depend on memory and network performance. A machine can rank high on HPL and still be slow for real science. HPCG measures the realistic case.

Important honesty point for the viva

Our project is **HPCG-inspired**, not the official HPCG. It keeps HPCG's essential ingredients (3D stencil matrix, sparse matrix-vector product, global dot products, halo exchange) but has no multigrid preconditioner and follows none of HPCG's official run rules. Our numbers are **not** HPCG or TOP500 results.

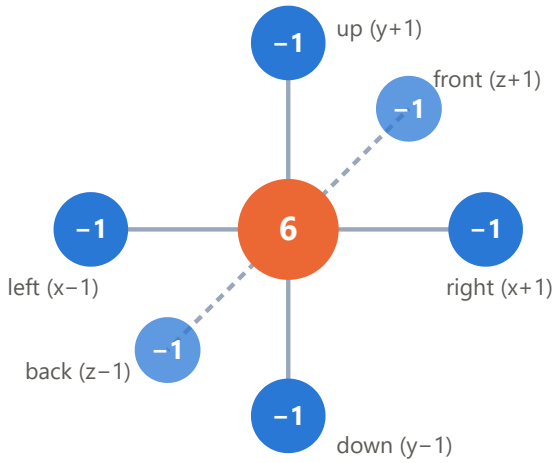
3 The problem we solve

We solve the **3D Poisson equation** (the model equation for heat, diffusion, pressure) on a grid of **nx × ny × nz** points. Each grid point gives one equation linking it to its 6 face neighbours — the **7-point stencil**:

$$6 \cdot x_{centre} - x_{left} - x_{right} - x_{up} - x_{down} - x_{front} - x_{back} = b$$

All equations together form **Ax = b**. For the 128 × 128 × 128 grid that is **2,097,152 equations** with **14,581,760 non-zeros**: more than **99.99%** of the matrix is zero — a **sparse matrix**.

We choose **b = A × 1**, so the exact answer is all ones. That lets us measure the true error of every run.



The 7-point stencil: 6 on the diagonal, −1 for each neighbour inside the grid. Neighbours outside the grid (the boundary) are dropped.

Why the matrix is safe to use with CG (SPD)

CG only works for **symmetric positive definite (SPD)** matrices. Ours is SPD because it is symmetric (if i is j's neighbour, j is i's neighbour), the diagonal (6) is at least the sum of the off-diagonals (6 × 1), strictly larger on boundary rows, and the grid is connected. The tests also confirm it numerically with a **Cholesky factorisation** (only succeeds for SPD) and by matching the exact smallest eigenvalue.

CSR storage — storing only the non-zeros

Example 4×4 matrix

6	-1	0	0
-1	6	-1	0
0	-1	6	-1
0	0	-1	6

```
values
[6,-1, -1,6,-1, -1,6,-1, -1,6]

col_idx
[0,1, 0,1,2, 1,2,3, 2,3]

row_ptr
[0, 2, 5, 8, 10]
```

Only the 10 non-zeros are stored.
Row i lives in values[row_ptr[i]] … values[row_ptr[i+1] − 1].

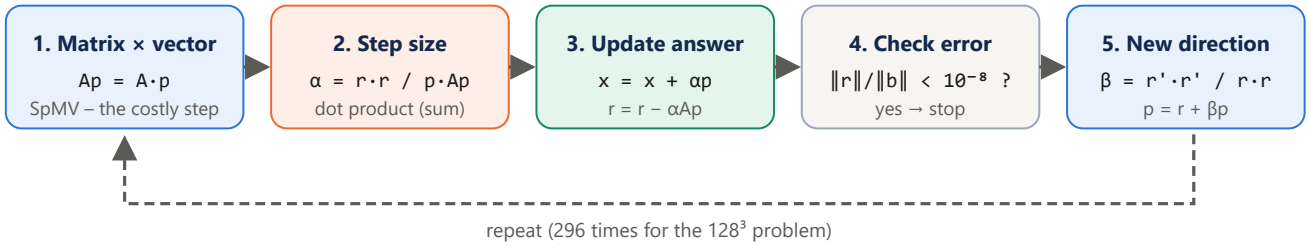
Compressed Sparse Row (CSR) uses three arrays: `values` (non-zeros row by row), `col_idx` (their columns) and `row_ptr` (where each row starts).

The sparse matrix-vector product (**SpMV**) then reads each non-zero once: `y[i] = Σ values[k] · x[col_idx[k]]`.

Column indices are stored as 4-byte integers (instead of 8) because SpMV is limited by memory traffic: 12 bytes per non-zero instead of 16 (−25%).

4 The Conjugate Gradient method

CG is the standard iterative method for large sparse SPD systems. It starts from a guess `x = 0`, computes the **residual** `r = b − Ax` (how wrong the guess is) and the search direction `p = r`, then repeats:



PER ITERATION

1 sparse matrix-vector product, 2 dot products, 3 vector updates.

STOPPING RULE

Stop when the relative residual ||r||/||b|| is below 10^{-8}.

SAFETY

If p·Ap ≤ 0 ("breakdown") or a NaN/infinity appears, the solver stops safely and reports it.

Why the number of iterations grows with the grid: the condition number κ of the matrix grows like (grid size)^2, and CG needs about √κ iterations, so iterations grow **linearly** with grid size. Measured: 41 iterations at 16^3, 158 at 64^3, 296 at 128^3, 368 at 160^3 — about 2.3–2.6 × N.

Counting work (FLOPs): each iteration costs `2·nnz + 10n` floating-point operations (2 per non-zero for SpMV, 2 per entry for each of the 5 vector operations). GFLOPS = operations ÷ (time × 10^9).

5 Four implementations

BASELINE

Sequential

One core, plain loops. Two pairs of loops are fused (SpMV + dot product; updates + dot product) so memory is read fewer times. Every other version uses exactly the same fusion, so comparisons are fair.

SHARED MEMORY

OpenMP

Each loop becomes `#pragma omp parallel for schedule(static)`. Dot products use `reduction(+:sum)` — each thread sums privately, results combined at the end — so there are no race conditions.

DISTRIBUTED MEMORY

MPI

Rows are split into contiguous blocks, one per process (uneven splits handled). Each process builds only its own rows. Vector data is exchanged before each SpMV; dot products use `MPI_Allreduce`.

BOTH

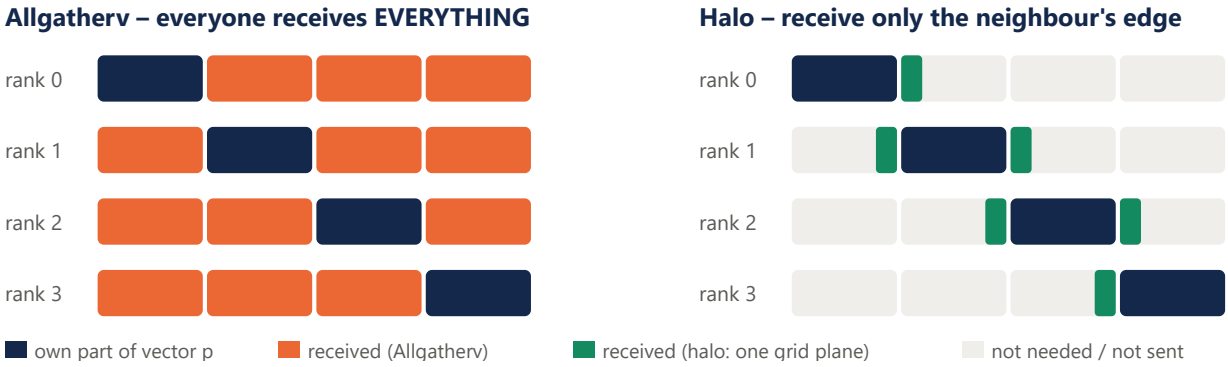
Hybrid MPI + OpenMP

MPI between processes, OpenMP threads inside each. Uses `MPI_THREAD_FUNNELED`: only the main thread calls MPI, outside parallel regions. The code checks the level at start-up.

> **Why all ranks stay in step (no deadlock)**
>
> Every decision — step size α, direction β, "stop or continue" — uses values from `MPI_Allreduce`, which are identical on every process. So every process makes the same sequence of MPI calls and leaves the loop at the same iteration. The tests check that the iteration counts match on every rank.

6 How MPI processes communicate

To multiply its rows by the vector **p**, a process needs some entries of p owned by other processes (its neighbours' rows). The project implements two ways of getting them:



Four MPI ranks each own a quarter of vector p. Left: Allgatherv copies the full vector to everyone. Right: halo exchange sends only one grid plane from each neighbour.

Scheme	What is sent each iteration	Data at 4 procs, 64^3	Comm. share at 8 procs, 128^3
Allgatherv (replicated vector)	Every process receives the whole vector	196,608 values	51%
Halo (`MPI_Isend` / `MPI_Irecv`)	**Only the neighbours' boundary plane**	**4,096 values**	**11.5%**

Both also need two `MPI_Allreduce` calls per iteration for the dot products — a global synchronisation that every process must reach.

7 How we measured

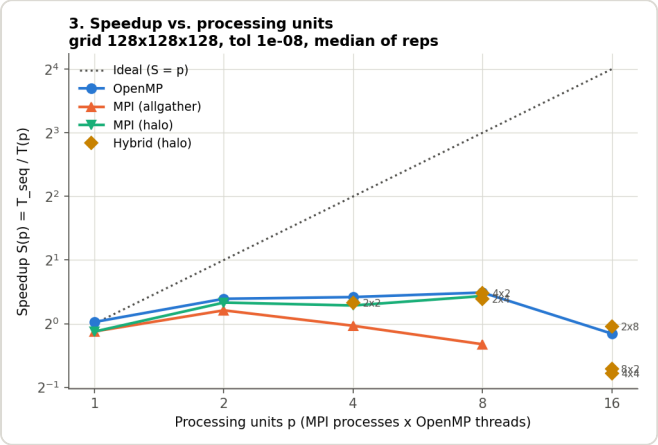
- **Same problem for everyone:** 128^3 grid, tolerance 10^{-8}, at most 5000 iterations. Every one of the 20 strong-scaling runs needed exactly **296 iterations** and reached the same accuracy.
- **Only the solve is timed** — not matrix generation or memory allocation.
- **1 warm-up + 5 timed runs**, report the **median** (robust to outliers).
- **MPI time = slowest process** (`MPI_MAX`), because the job ends when the last one finishes.
- **Speedup** $S(p) = T_{sequential} / T(p)$.
- **Efficiency** $E(p) = S(p) / p \times 100\%$, with p = processes × threads.
- **Communication share** = (exchange + Allreduce time) / total time.
- **Memory bandwidth** measured separately with a STREAM-style triad test (`a = b + s·c`).
- Configurations needing more cores than available are skipped and logged — nothing is invented.

Environment: Intel Core 7 240H laptop (16 logical CPUs), Windows 11, Ubuntu 24.04 Docker container, g++ 13.3 (-O3), OpenMP 4.5, Open MPI 4.1.6.

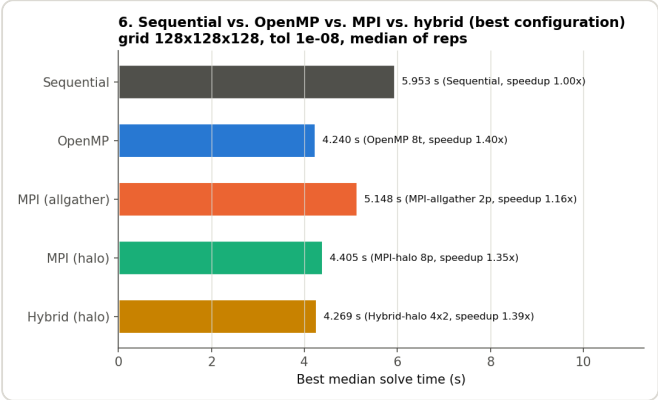
8 Results

Strong scaling — 128^3 grid, 2,097,152 equations

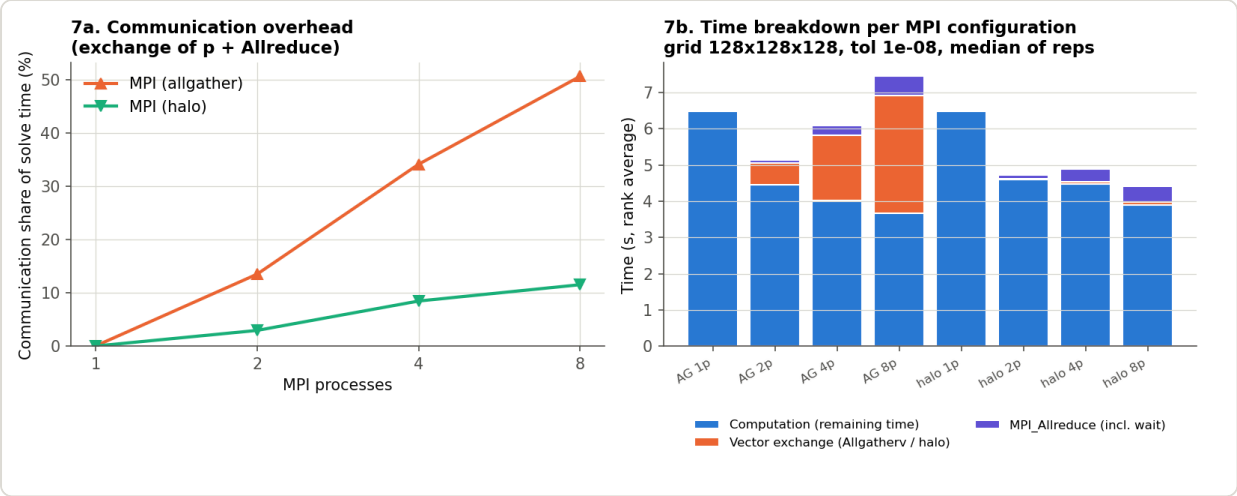
Configuration	Units p	Time (s)	Speedup	Efficiency	GFLOPS	Comm. share
Sequential	1	5.953	1.00	100%	2.50	–
OpenMP 2 threads	2	4.540	1.31	65.6%	3.28	–
OpenMP 4 threads	4	4.455	1.34	33.4%	3.34	–
OpenMP 8 threads (best)	**8**	**4.240**	**1.40**	**17.5%**	**3.51**	**–**
OpenMP 16 threads	16	6.634	0.90	5.6%	2.24	–
MPI Allgatherv 8 procs	8	7.446	0.80	10.0%	2.00	50.6%
MPI halo 8 procs	8	4.405	1.35	16.9%	3.38	11.5%
Hybrid 4 procs × 2 threads	8	4.269	1.39	17.4%	3.49	5.1%
Hybrid 4 × 4	16	10.229	0.58	3.6%	1.45	45.9%



Speedup. The dotted line is ideal (S = p). Every version flattens near 1.4× — the memory wall.

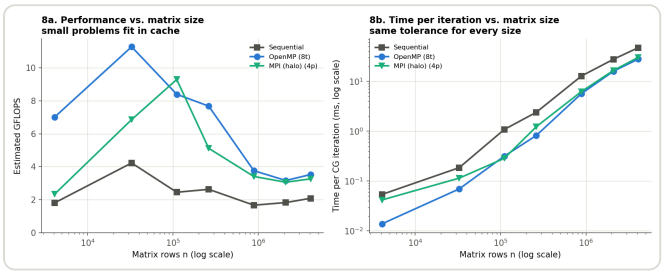


Best of each version. OpenMP, MPI-halo and hybrid all reach ≈ 4.2–4.4 s; Allgatherv lags.



Communication. Left: communication share grows quickly for Allgatherv (orange) but stays low for halo (green). Right: time split into computation, vector exchange and Allreduce.

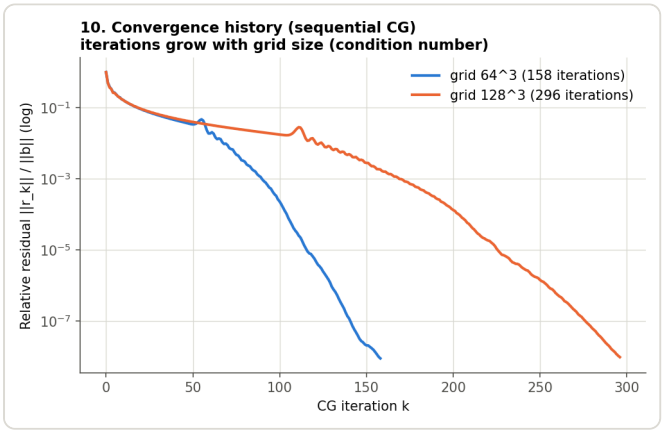
Problem size and the cache effect



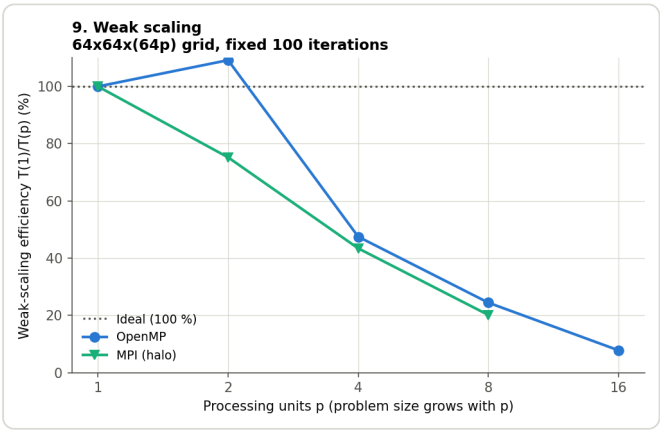
Performance vs size. Small problems fit in the CPU cache and look 2–4× faster; from 96^3 upward data comes from main memory.

Grid	Iterations	OpenMP 8t speedup
16^3	41	3.89
32^3	81	2.67
48^3	120	3.43
64^3	158	2.92
96^3	225	2.26
128^3	296	1.73
160^3	368	1.69

Lesson: benchmarks must use problems far larger than the cache — which is exactly what HPCG requires.



Convergence. The residual falls below 10^{-8}; bigger grids need more iterations.



Weak scaling. Problem grows with cores; on one laptop the shared memory bus makes time grow almost linearly.

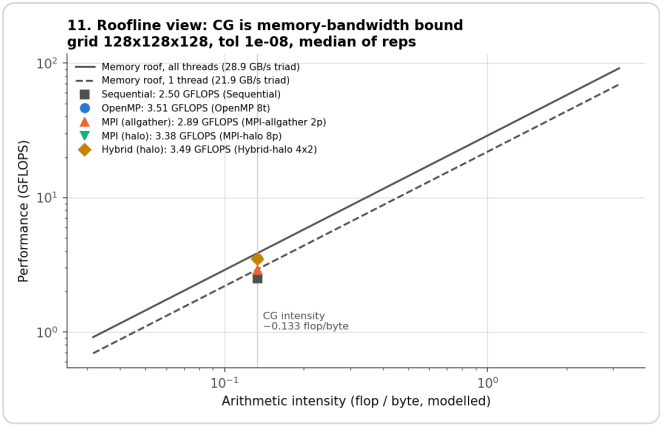
9 Why the speedup is limited

Arithmetic intensity = operations ÷ bytes moved. Per CG iteration we do `2·nnz + 10n` operations but move about `12·nnz + 96n` bytes, giving **≈ 0.13 operations per byte** — very low.

The **roofline model** says: attainable performance = min(peak FLOPS, intensity × bandwidth). At 0.13 flop/byte the bandwidth line is the limit:

```
0.133 × 28.9 GB/s ≈ 3.8 GFLOPS
```

We measured **3.51 GFLOPS** — about **90%** of the hardware limit. The code is efficient; the memory is the bottleneck.



Roofline. All implementations sit just under the measured memory roof.

Threads	1	2	4	8	16
Triad bandwidth (GB/s)	21.9	24.1	27.4	28.9	27.8

Not Amdahl's law

Amdahl limits speedup by a serial fraction. Here almost nothing is serial — the shared memory bus is the limit.

16 threads slower than 8

16 logical CPUs share fewer physical cores (hyper-threading, P/E cores). Extra threads add no bandwidth and compete.

On a real cluster

Each node brings its own memory bus, so MPI across nodes scales better — then network latency becomes the limit.

10 Tools, tests & files

Area	What we used / where it is
Language & build	C++17, g++ 13.3 (-O3), CMake + CTest
Parallelism	OpenMP 4.5, Open MPI 4.1.6
Analysis	Python 3 — Pandas, NumPy, Matplotlib (11 graphs, HTML dashboard)
Environment	Docker Ubuntu 24.04 container (the Windows laptop had no compiler/MPI)
Matrix & CSR	`src/matrix/matrix_generator.cpp` , `src/matrix/csr_matrix.cpp`
Solvers	`src/sequential/` , `src/openmp/` , `src/mpi/` , `src/hybrid/`
Benchmark runner	`src/benchmark/benchmark.cpp` , `scripts/run_benchmarks.sh`
Tests	`tests/` — 10 CTest targets incl. MPI on 1, 2, 3, 4 processes
Results	`results/benchmark.csv` , `results/summary.md` , `results/graphs/` , `results/dashboard.html`

Report	`report/project_report.md` (24 sections, 21 references)

What the 10 tests prove: the matrix is symmetric positive definite; CSR and SpMV are correct; CG reaches the exact answer on known systems; breakdown, NaN and zero right-hand side are handled; OpenMP, MPI and hybrid give the same answer as sequential (within 10^{-8}); MPI works with uneven row splits; every rank stops at the same iteration.

11 Limitations & future work

LIMITATIONS

- One laptop: MPI messages go through shared memory, not a real network.
- Not official HPCG: no multigrid preconditioner, 7-point stencil, 1D row split.
- Laptop + virtual machine add timing noise (thermal throttling, background apps).
- No hardware counters (`perf` unavailable in Docker) — memory traffic is modelled.

FUTURE WORK

- Add the multigrid / Gauss-Seidel preconditioner and 27-point stencil.
- 3D domain decomposition for smaller halos.
- Overlap communication with computation; pipelined CG.
- Run on a real multi-node cluster; add a GPU (CUDA) version.

12 Glossary

Term	Meaning
Sparse matrix	Mostly zeros; only non-zeros are stored (ours: > 99.99% zeros).
nnz	Number of non-zero entries (14.58 million at 128^3).
SPD	Symmetric positive definite: $A = A^T$ and $x^TAx > 0$. Required for CG.
CSR	Compressed Sparse Row storage: values, col_idx, row_ptr.
SpMV	Sparse matrix × vector — the most expensive step of CG.
Residual / relative residual	$r = b - Ax$ (how wrong the answer is); $\|r\|/\|b\|$ is scale-free. We stop at 10^{-8}.
Thread / process	Thread: shares memory with others in one program. Process: independent program with private memory.
Reduction	Combining partial results (e.g. sums) from all threads/processes into one value.
Race condition	Two threads writing the same data at once → wrong result. Avoided by design.
MPI_Allreduce	Every process contributes a value, all receive the combined result.
Allgatherv	Every process receives every other process's block → whole vector replicated.
Halo exchange	Each process receives only the neighbouring boundary values it needs.
MPI_THREAD_FUNNELED	Threads allowed, but only the main thread calls MPI.
Speedup / efficiency	$S = T_{seq}/T(p)$; $E = S/p \times 100\%$.

Strong / weak scaling	Strong: same problem, more cores. Weak: problem grows with cores.
Memory bandwidth	GB/s the memory system can deliver; measured with the STREAM triad.
Arithmetic intensity	Operations per byte moved; CG ≈ 0.13 (low → memory-bound).
Roofline model	Performance ≤ min(peak FLOPS, intensity × bandwidth).
Amdahl's law	Speedup ≤ 1 / (s + (1−s)/p) for a serial fraction s (strong scaling).
Gustafson's law	With problem size growing with p (weak scaling), scaled speedup = s + p(1−s).
Cache-resident	Data fits in fast CPU cache → unrealistically good speed for small problems.

13 Numbers to memorise

2,097,152
equations (128^3)

14.58 M
non-zeros

296
iterations, every version

10^{-8}
tolerance

5.95 s
sequential time

4.24 s
best (OpenMP 8 threads)

1.40×
best speedup

1.32×
bandwidth growth 1→8 threads

21.9 → 28.9
GB/s triad bandwidth

0.13
flops per byte

51% vs 11.5%
comm. share: Allgatherv vs halo

1.69×
halo faster than Allgatherv

~90%
of memory roof reached

10 / 10
tests pass

50
configurations, 0 failed

16
logical CPUs (Core 7 240H)

14 Viva questions & answers

Answer in your own words — the key point is in bold. If you are unsure: "I'm not certain; my understanding is …, and I would verify it in the report / code." Honesty scores better than guessing.

A. Basics and motivation

Q1 What is your project in one line?

An HPCG-inspired benchmark that solves a large sparse system with the Conjugate Gradient method using sequential, OpenMP, MPI and hybrid versions, and analyses **why sparse solvers don't scale like dense ones**.

Q2 What is HPC?

Using many processors together to solve problems faster or to solve bigger problems, on clusters of multicore nodes connected by a fast network.

Q3 What is HPL? Why is it not enough?

HPL solves a *dense* linear system by LU factorisation and ranks the TOP500. It is **compute-bound**: lots of arithmetic per byte of data. Most real scientific codes are **memory- and communication-bound**, so HPL overstates how fast machines are for them.

Q4 What is HPCG?

The High Performance Conjugate Gradient benchmark (Dongarra, Heroux and Luszczek, 2013–2016). It solves a sparse 3D-stencil system with CG preconditioned by multigrid. It stresses **memory bandwidth and network latency**, like real applications. Top machines reach only a few percent of peak on HPCG.

Q5 Is your project the official HPCG?

No. It is HPCG-*inspired* and educational. Official HPCG uses a 27-point stencil, a multigrid preconditioner, a 3D domain decomposition and strict run rules. We use a 7-point stencil, plain CG and a 1D row split. Our numbers are not HPCG or TOP500 results.

Q6 Why did you choose this topic / what is new?

It shows experimentally *why* HPCG was needed. We added a direct comparison of two MPI communication schemes and a **roofline analysis with measured bandwidth**, which explains the low speedup.

B. Mathematics

Q7 What equation are you solving?

The 3D Poisson equation, discretised on a grid with a 7-point stencil. Each grid point gives one equation, `6·u(centre) − Σ u(neighbours) = b`. In matrix form that is `Ax = b`.

Q8 What is a sparse matrix? How sparse is yours?

A matrix that is mostly zeros, so only the non-zeros are stored. Ours has at most 7 non-zeros per row out of 2 million columns, so **more than 99.99 % zeros**.

Q9 What is CSR format?

Compressed Sparse Row uses three arrays:

- `values` : the non-zeros, row by row;
- `col_idx` : the column of each non-zero;
- `row_ptr` : where each row starts.

It saves memory and lets SpMV read the data sequentially.

Q10 What is SPD and why does it matter?

Symmetric ($A = A^T$) and positive definite ($x^TAx > 0$ for every non-zero x). **CG is only guaranteed to converge for SPD matrices.**

Q11 How do you know your matrix is SPD?

Proof: it is symmetric, has a positive diagonal and is diagonally dominant ($6 \geq$ sum of six 1s), strictly so on boundary rows, and the grid is connected, which together imply SPD. Tests check it too: a **Cholesky factorisation succeeds** (that only works for SPD matrices), and the smallest eigenvalue matches its exact formula.

Q12 Explain the Conjugate Gradient algorithm.

Start with a guess x_0 and residual $r = b - Ax$ (the error signal), and set the search direction $p = r$. In each iteration:

1. $\alpha = (r \cdot r)/(p \cdot Ap)$, the step size;
2. $x = x + \alpha p$;
3. $r = r - \alpha Ap$;
4. $\beta = (r_{new} \cdot r_{new})/(r \cdot r)$;
5. $p = r + \beta p$.

Stop when $||r||/||b|| < 10^{-8}$. Each step uses **one matrix-vector product, two dot products and three vector updates**.

Q13 Why does CG work? What is special about it?

It chooses search directions that are A-conjugate, so it never undoes earlier progress. It minimises the error over a growing Krylov subspace. In exact arithmetic it finishes in at most n steps; in practice it gets close much sooner.

Q14 What is the residual? What is the relative residual?

$r = b - Ax$ measures how wrong the current answer is. The relative residual is $||r||/||b||$, which doesn't depend on scale. We stop at 10^{-8}.

Q15 Why does the iteration count grow with grid size (41 → 368)?

The condition number κ grows like (grid size)^2, and CG needs roughly $\sqrt{\kappa}$ iterations, so **iterations grow linearly with grid size**. Our data confirms it: about 2.3–2.6 × N.

Q16 Why b = A × 1?

Then the **exact answer is known** (all ones), so we can measure the true error, not just the residual. HPCG uses the same trick.

Q17 What happens if the matrix is not SPD?

p·Ap can become ≤ 0. The code detects this ("breakdown") and stops safely. It also detects NaN or infinity. These cases are tested.

C. Parallel programming

Q18 Difference between OpenMP and MPI?

OpenMP uses **threads in one process sharing memory**, parallelised with `#pragma omp` directives. MPI uses **separate processes with private memory** that exchange explicit messages, and it can run across many computers.

Q19 How did you parallelise CG with OpenMP?

Each loop (SpMV, vector updates) is `#pragma omp parallel for schedule(static)`. Dot products use `reduction(+:sum)`, so each thread sums privately and the results are combined at the end. There are **no race conditions**.

Q20 What is a race condition? How did you avoid it?

Two threads updating the same variable at the same time, giving a wrong result. We avoid it because each thread writes **only its own rows**, and dot products use reductions instead of a shared counter.

Q21 What is a reduction?

Combining partial results from all threads (or processes) into one value, for example a sum. OpenMP uses `reduction(+:x)`; MPI uses `MPI_Allreduce`.

Q22 Why static scheduling?

All iterations cost the same, and static scheduling gives each thread the **same rows in every loop**, which keeps its data in its cache.

Q23 How is the work divided in MPI?

The rows are split into contiguous blocks, one per process. If the division is uneven, the first (n mod P) processes get one extra row. **Each process generates only its own rows**, so no process holds the whole matrix.

Q24 Why do MPI processes need to communicate in CG?

1. SpMV for a row needs vector values from neighbouring rows, which may belong to another process. That needs an exchange.
2. Dot products need the **sum over all processes**, which needs `MPI_Allreduce`, twice per iteration.

Q25 What is MPI_Allreduce?

A collective operation: every process contributes a value, the values are combined (summed), and **every process receives the result**. We use it for α and β so that all processes agree.

Q26 Allgatherv vs halo exchange?

- **Allgatherv**: every process gets the *entire* vector each iteration. It is simple but sends O(n) data per process, so it doesn't scale.
- **Halo**: each process gets only the boundary values (one grid plane from each neighbour) using `MPI_Isend` / `MPI_Irecv`.

At 4 processes on 64^3 that is 196,608 values versus 4,096.

Q27 Why did MPI with 8 processes using Allgatherv get slower than sequential?

Communication took **51 % of the time**. Each process received about 1.8 million values (14.7 MB) every iteration, which cost more than the computation saved.

Q28 How do you make sure MPI processes don't deadlock?

Every decision (stop or continue, α, β) uses values from `MPI_Allreduce`, which are **identical on all processes**. So all processes run the same sequence of collective calls. The tests check that the iteration counts are equal on every rank.

Q29 What is hybrid MPI + OpenMP? Why use it?

MPI between processes (or nodes) and OpenMP threads inside each process. It means fewer processes, which gives fewer messages and less duplicated memory, and it uses shared memory within a node. On real clusters this is the standard approach.

Q30 What is MPI_THREAD_FUNNELED?

A thread-support level where the program can have threads but **only the main thread calls MPI**. Our hybrid code calls MPI only outside OpenMP regions. It requests this level with `MPI_Init_thread` and checks it.

Q31 Why `--bind-to none`?

By default Open MPI pins each process to one core. In hybrid mode that would force all of a process's threads onto **one core**. `--bind-to none` lets the operating system spread them out.

D. Performance analysis

Q32 Define speedup and efficiency.

- Speedup S(p) = T(sequential) / T(p).
- Efficiency E(p) = S(p)/p × 100 %, where p = processes × threads.

Example: 8 threads gave S = 1.40, so E = 17.5 %.

Q33 Why is your speedup only 1.4× on 8 cores?

Memory bandwidth. CG does about 0.13 flops per byte, so it is limited by how fast memory delivers data. We measured bandwidth growing only 1.32× from 1 to 8 cores, and the speedup (1.40×) matches it. The cores are waiting for memory.

Q34 Is low efficiency a sign of bad code?

No. The roofline shows we reach about 90 % of the measured memory bandwidth. The code is near the hardware limit; the hardware is the bottleneck. This is exactly what HPCG demonstrates.

Q35 What is the roofline model?

Attainable performance = min(peak FLOPS, arithmetic intensity × memory bandwidth). At low intensity (CG: 0.13 flops per byte) the bandwidth line is the limit: 0.133 × 28.9 GB/s ≈ 3.8 GFLOPS. We measured 3.5.

Q36 What is arithmetic intensity? How did you get 0.13?

Flops divided by bytes moved. Per iteration we do 2·nnz + 10n flops and move about 12·nnz + 96n bytes (12 bytes per non-zero: an 8-byte value and a 4-byte index; 96 bytes per row for the vectors). With about 7 non-zeros per row that gives about 0.13.

Q37 What is STREAM triad?

A standard memory-bandwidth test: `a[i] = b[i] + s·c[i]` over big arrays, counting 24 bytes per element. We use it to measure the bandwidth "roof".

Q38 What are Amdahl's and Gustafson's laws?

- **Amdahl:** with a serial fraction s, speedup ≤ 1/(s + (1−s)/p), the limit for a *fixed* problem size (strong scaling).
- **Gustafson:** if the problem grows with p (weak scaling), the scaled speedup can keep growing.

Our limit is neither of these: it is the shared memory bandwidth.

Q39 Strong scaling vs weak scaling?

- **Strong:** the same problem, more cores, with the goal of less time.
- **Weak:** the problem grows with the cores, with the goal of the same time.

We did both. Weak scaling fails on one laptop because all cores share one memory bus; on a cluster each node brings its own memory.

Q40 Why is 16 threads slower than 8?

The laptop has 16 *logical* CPUs but fewer physical cores (hyper-threading, plus a mix of performance and efficiency cores). Extra hyper-threads add **no memory bandwidth** and compete with each other, and with the VM and Windows.

Q41 Why are small problems faster per equation (up to 3.9× speedup)?

They **fit in the CPU cache**, which is much faster than main memory. That gives misleadingly good numbers, so benchmarks like HPCG require problems much larger than the cache. Our main problem is 263 MB.

Q42 What is GFLOPS and how do you calculate it?

Billions of floating-point operations per second: `GFLOPS = (counted operations) / (time × 10⁹)`, with 2·nnz + 10n operations per iteration. Communication is not counted as useful work.

Q43 How did you measure time fairly?

- The timer covers only the solve, not matrix generation.
- There is 1 warm-up run, then 5 timed runs, and we report the **median**.
- For MPI we report the **slowest process** (MPI_MAX).
- Every version solves the same problem to the same tolerance, in the same 296 iterations.

Q44 Why the median, not the average?

The median is not affected by outliers. One slow run, for example caused by a background process, doesn't distort it.

Q45 Why does MPI with 1 process take longer than sequential (6.48 s vs 5.95 s)?

It is the overhead of the distributed code path: copying into the exchange buffer, bookkeeping and 592 `MPI_Allreduce` calls (two per iteration). It is small and constant.

E. Implementation, testing, tools

Q46 Which tools and languages did you use, and why?

- **C++17:** fast, with direct control of memory.
- **OpenMP and Open MPI:** the industry standards.
- **CMake:** portable builds.
- **Python, Pandas and Matplotlib:** analysis and graphs.
- **Docker:** the Windows laptop had no compiler, so a Linux container makes the setup reproducible anywhere.

Q47 How do you know your results are correct?

- 10 automated tests: matrix properties, the exact solution, and parallel results matching sequential.
- Every benchmark run checks the true residual and the error from the exact answer (about 7.7×10^{-8}).
- All 50 configurations converged with the same 296 iterations.

Q48 Why compare with a tolerance and not exact equality?

Floating-point addition isn't associative, so parallel sums in a different order give slightly different last digits. We even observed OpenMP giving bit-different results between identical runs. We compare within 10^{-8}.

Q49 Why store column indices as 32-bit integers?

SpMV is memory-bound, and 4-byte indices instead of 8-byte cut the matrix traffic by 25 % (12 instead of 16 bytes per non-zero).

Q50 What is kernel fusion?

Combining two loops into one, for example SpMV together with its dot product, so the data is read from memory once instead of twice. All four versions use the same fusion, so the comparison is fair.

Q51 What are the limitations?

1. Single laptop: MPI communication goes through shared memory, not a real network.
2. No multigrid preconditioner, a 7-point stencil and a 1D split, so it is not official HPCG.
3. The laptop and VM add timing noise.
4. No hardware counters (`perf` doesn't work in Docker), so memory traffic is modelled.

Q52 Future work?

- Add the multigrid preconditioner and the 27-point stencil.
- Use a 3D domain decomposition.
- Overlap communication with computation.
- Run on a real multi-node cluster.
- Add a GPU version (CUDA).
- Measure hardware counters on bare metal.

Q53 What would change on a real supercomputer?

Each node has its own memory bandwidth, so MPI across nodes would scale much better (weak scaling). The network latency of `MPI_Allreduce` would then become the main bottleneck. The halo approach would matter even more.

Q54 What did you personally learn?

(Make this personal.) For example: that parallel speed depends on memory and communication, not just core count; how to measure performance fairly; and how to verify parallel code against a sequential baseline.
